TBR-QL: Q-Learning Enhanced Trust-Based Routing for Secure Underwater Wireless Sensor Networks

Abhishek Jain, Krish, Sujal Goel
Department of Information Technology
Netaji Subhas University of Technology
New Delhi, India

***Abstract*—Routing in underwater wireless sensor networks (UWSNs) differs fundamentally from terrestrial wireless networks due to the use of acoustic waves for inter-node communication, which causes lower bandwidth, high propagation delays, and severe multipath fading compared to radio frequency transmission. These characteristics make UWSNs particularly vulnerable to insider attacks from compromised nodes. To address this vulnerability, this article proposes TBR-QL, a Q-learning enhanced trust-based unicast routing protocol that combines historical node behavior with accumulated routing experience to make informed relay selection decisions. Each node maintains a Q-value for every neighbor, updated continuously from observed forwarding outcomes and incorporating factors such as residual energy, depth advancement towards sinks, propagation delay, enabling the protocol to distinguish reliable neighbors from malicious ones over time. Simulation results show that TBR-QL consistently outperforms the baseline TBR [10] across various attacks while maintaining stable performance under elevated network loads.**

***Index Terms*—Network security, reinforcement learning, routing protocol, trust management model, underwater wireless sensor networks (UWSNs).**

I. Introduction

More than 70% of the Earth's surface is covered by oceans, yet less than 5% of seabed has been explored. Underwater wireless sensor networks(UWSNs) have become important for closing this gap. They enable applications like ocean exploration, monitoring the environment, detecting seismic activity, and surveying ports [1], [2].In these systems, sensor nodes placed at various depths collect environmental data and send it to surface sink nodes through multi-hop acoustic communication.

Operating underwater is very different from wireless networking on land. Radio frequency signals experience severe attenuation in water, making acoustic communication the only practical option for long-distance underwater transmission [3]. However, acoustic channels have very limited bandwidth resulting in data rates of only a few kilobits per second, large propagation delays due to the speed of sound in water (approximately 1500 m/s), as well as severe multipath fading effects [4]. On top of this, underwater sensor nodes are battery-powered which generally cannot be recharged or replaced after deployment, requiring networks to operate autonomously for long periods of time under strict energy constraints.

The security aspects add further complexity. Compromised insider nodes keep valid network credentials allowing them to bypass all conventional cryptographic defenses such as key-based authentication [5]. Such nodes can silently drop transmitted packets, create intentional transmission delays or flood the channel with repeated packet copies. The damage from these attacks is amplified due to the nature of acoustic communication as discussed above, making security essential for reliable UWSNs operation.

Trust management has emerged as the primary framework for addressing insider attacks in UWSNs, where protocols monitor node behavioral history and assign trust scores to progressively exclude malicious nodes from routing decisions [6]–[9]. Liu et al. [10] proposed TBR, which directly combines a trust model with depth-based relay selection and demonstrates strong performance against individual attack types. However TBR's relay selection relies entirely on an instantaneous routing metric recomputed at each forwarding step, which cannot distinguish between a malicious node and a congested legitimate node when both exhibit similar trust patterns under higher network loads as discussed in later sections.

This article proposes TBR-QL, which integrates Q-learning directly into the TBR routing framework. Each node maintains a learned Q-value for every neighbor, updated continuously from observed packet forwarding outcomes. Rather than relying solely on the current state of a neighbor at each routing step, TBR-QL accumulates routing experience over time, allowing the relay selection to improve progressively and distinguish reliable neighbors from malicious or congested ones as the network operates. The main contributions of this paper are as follows:

1) A Q-learning enhanced relay selection mechanism where each node maintains a per-neighbor Q-value updated from rewards incorporating trust, residual energy, propagation delay, and depth advancement toward the sink.
2) An asymmetric learning rate design with a faster punishment rate and slower recovery rate, ensuring malicious nodes are penalized rapidly while preventing on-off attackers from quickly regaining high Q-values during cooperative phases.
3) An autonomous on-off attack model where relay nodes randomly enter and exit attack phases without pre-

assignment, representing a realistic insider threat specifically designed to evade trust-based detection.

The remainder of this article is organized as follows. Section II reviews related work. Section III describes the system model. Section IV presents the design of TBR-QL. Section V provides simulation results. Section VI concludes the article while Section VII discusses future research directions.

II. Related Work

Research on secure routing in UWSNs has developed along two directions: trust management models that evaluate node behavior, and reinforcement learning-based approaches that adapt routing decisions from experience.

Han et al. [6] proposed ARTMM, an attack-resistant trust model based on multidimensional trust metrics that considers packet delivery rate, energy consumption, and data content to evaluate node trustworthiness. While widely referenced, it does not integrate trust with routing design, meaning the network may still route through partially trusted malicious nodes. Jiang et al. [11] introduced a dispute-adjudication-based trust mechanism with a game-theoretic incentive scheme to encourage honest recommendations, but similarly pays insufficient attention to relay node screening during forwarding. Du et al. [7] proposed ITrust, which uses insolation forests to detect anomalous behavior without requiring explicit attack pattern definitions, though its performance is sensitive to parameter settings. Su et al. [8] proposed a redeemable SVM-DS fusion mechanism that classifies node trust and allows recovery based on historical environment information, but shares the same parameter sensitivity limitation. Signori et al. [9] proposed a channel-based trust mechanism using Hidden Markov models to characterize acoustic channel behavior, though reliability suffers under the variable conditions of the underwater environment.

Liu et al. [10] proposed TBR, which address the limitation shared by the above works by directly combining trust management with depth-based relay selection. TBR evaluates each neighbor across interaction, energy, information, and delay trust dimensions and selects the next hop using a composite score incorporating trust, residual energy, and inter-node distance, demonstrating strong performance over both ARTMM and the classical DBR protocol [12]. But when network load increases, TBR is not able to differentiate between packet dropping and delay by malicious nodes and congested nodes which results in TBR penalizing congested nodes, thus degrading performance.

On the reinforcement learning side, Li et al. [13] proposed SDN-QLTR, a Q-learning-assisted trust routing scheme for software-defined UASNs. While effective, its centralized SDN controller is unsuitable for fully distributed deployments where nodes must make independent routing decisions. Hosseinzadeh el al. [14] proposed QLTM, which applies Q-learning to trust parameter updates rather than to the relay selection decision itself, improving trust computation but does not directly decide the next hop.

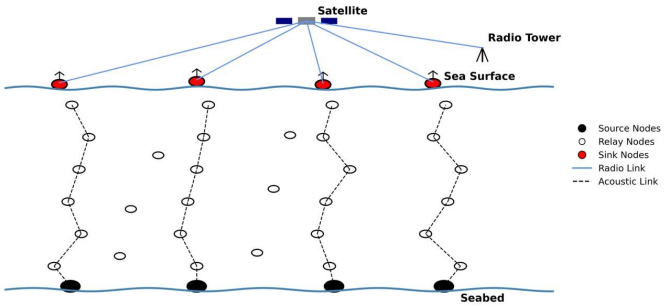

Fig. 1. Underwater Wireless Sensor Network Topology

In summary, existing trust-based approaches evaluate node behavior effectively but select relay nodes based on instantaneous scores without learning from past routing outcomes. Reinforcement learning has been applied to UWSN routing but either depends on centralized control or improves trust computation rather than the relay selection itself. TBR-QL bridges this gap by combining the trust framework of TBR with Q-learning driven relay selection, where each routing decision is informed by accumulated routing experience rather than the current state of a neighbor alone.

III. Preliminaries

A. Network Model

The network model follows the same deployment assumptions as TBR [10]. As illustrated in Fig. 1, the network consists of three tiers: source sensor nodes at the lower depth that sense and generate application data, intermediate relay nodes that carry packets upward through the network, and surface sink nodes that serve as the final collection points. Data travels from the bottom upward, one hop at a time, with each relay node passing the packet to a neighbor at a strictly shallower depth until it reaches the surface. This depth-constraint ensures that packets always make progress toward the sink rather than circulating within the network.

Each node knows its own three-dimensional position and those of neighbors within communication range R, with neighbor information discovered through periodic hello broadcasts. Trust values are initialized at network startup, and every node participates in the trust management process throughout the simulation.

B. Attack Model

Four categories of insider attacks are considered in this article.

In a packet drop attack [10], a compromised node silently discards received packets rather than forwarding them, thus reducing the packet delivery ratio of the network. In a replay attack [10], the compromised node holds packets for a deliberate back-off period before forwarding, thus increasing end to end delay of the network. In a flooding attack [10], the node retransmits the same received packet multiple times, consuming channel bandwidth and draining the energy of

neighboring nodes. The mixed attack scenario combines all three attacks, with different subsets of the attacker population executing each attack simultaneously.

This article additionally incorporates a mixed on-off attack model as a more realistic representation of insider threat behavior. Unlike continuous attackers whose trust scores collapse predictably over time, an on-off attacker alternates between malicious and cooperative phases without any fixed schedule. Each compromised relay node makes an independent probabilistic decision upon receiving a data packet: with a trigger probability of 0.5 it enters an attack phase and drops all packets for 30 seconds, after which it resumes normal forwarding until the next packet reception triggers the same decision again. No coordination exists between attacking nodes - each operates entirely on its own.

What makes this difficult to defend against is the trust recovery that happens during cooperative phases. A node that drops packets for 30 seconds and then forwards correctly for an extended period gradually rebuilds its interaction history and can maintain a trust value that keeps it above the exclusion threshold. It can therefore re-enter routing decision before the protocol has fully penalized it. Continuous attackers do not have this option - their trust declines monotonically and exclusion is only a matter of time. The on-off attacker exploits the trust system's own recovery mechanism to stay in play.

IV. Design of TBR-QL

A. Overview of TBR-QL

TBR-QL is a secure routing protocol that extends TBR [10] with a Q-learning based relay selection mechanism. The protocol operates in three sequential phases as written below.

In the first phase, every node broadcasts hello packets to neighbors within its communication range. Each node uses the received information : position, energy and node identity, to build its neighbor table (Table I) and assign an initial trust value and Q-value to each discovered neighbor. Since no interaction history exists at this point, trust metrics Interaction Trust (T_s), Delay Trust (T_r) and Transmission Trust (T_t) are set to their neutral prior values, and the Q-value for each neighbor is seeded from the Ph (eq. 11) composite score. This gives the Q-table a reasonable starting point before any packet has been exchanged.

In the second phase, when a data packet needs to be forwarded, the node scans its neighbor table and applies two conditions: the candidate must be at a strictly shallower depth than the current node so the packet always moves toward the surface, and its integrated trust (T_G) must meet or exceed the trust threshold so that a lower trust nodes are will never got selected. Among the neighbors that pass both conditions, the node picks the one with the highest Q-value as the next hop and forwards the packet.

In the third phase, the node observes the outcome of each forwarding decision and updates both the trust metrics and the Q-value of the selected neighbor. A successful forward triggers an upward Q-value update using a positive reward. A dropped packet triggers a sharp downward Q-value update using a higher learning rate, and interaction trust (T_s) also reduced. If the node overhears repeated copies of the same packet arriving from a neighbor, indicating a Flooding attack, the transmission trust (T_t) is reduced proportionally and a negative Q-value update is applied to that neighbor. When a node receives a acknowledgment packet later then the expected propagation time, T_r is reduced to reflect the anomalous latency. Over time, this joint update of trust metrics and Q-values causes the relay selection to naturally converge away from malicious neighbors without requiring any explicit attack classification as shown in Fig. 6.

B. Trust Model

TBR-QL adopts the trust management framework of TBR [10] as its foundation, retaining the core trust aggregation structure through direct and indirect trust. Two modifications are introduced in this work with respect to the original TBR design trust model. First, the information trust metric of TBR is replaced in TBR-QL by transmission trust T_t. This trust value provides a more direct and computationally lightweight measure of flooding behavior, as each additional repeat immediately drives T_t toward zero without requiring a separate packet classification step. Second, the integrated trust T_G which in TBR is used to calculate Ph score (eq. 11) but here it is used as reward signal for the Q-learning update mechanism as described in eq. 15, extending its role to actively guide the reinforcement learning process. The trust management process otherwise follows the same procedure as TBR [10] and is not reproduced here for brevity. The key formulas are presented below for completeness and to establish the notation used in subsequent sections.

1) Interaction Trust(T_s): Interaction trust measures the reliability of packet forwarding of neighbor based on the history of successful and unsuccessful forwarding events observed by the current node. The trust is computed as [10]:

$$T_a^{ij} = \frac{a_{ij}}{a_{ij} + b_{ij}} \tag{1}$$

where a_{ij} is the number of packets successfully forwarded by node j and b_{ij} is the packets dropped by node j after receiving these packets from node i. When no interactions have occurred, T_a^{ij} is initialized to default as a neutral prior. To increase the sensitivity to malicious dropping behavior, a penalty factor δ is introduced:

$$\delta = \frac{1}{b^{ij} + 1} \tag{2}$$

The penalty factor ensures that each additional drop event causes a rapid and compounding reduction in trust. The formula for interaction trust is [10]:

$$T_s^{ij} = \delta \cdot T_a^{ij} \tag{3}$$

Even if a node drops a few packets, it incurs a sharp collapse in T_s^{ij} regardless of its historical success rate, making the metric highly sensitive to packet drop attacks.

2) Delay Trust (T_r): Delay trust captures delay attacks by comparing the actual forwarding latency of neighbor j against a theoretical delay computed from the measured inter-node distance. The theoretical delay is defined as:

$$t_{theoretical} = \frac{2 \cdot L_{ij}}{V_s} + c \tag{4}$$

where L_{ij} is the distance between node i and neighbor j, V_s = 1500 m/s is the speed of sound in water, and c is a constant added to account for processing and queuing overhead at each node. The factor of 2 reflects the round-trip propagation time,i.e., the time for the packet to travel from node i to neighbor j and for the acknowledgment to return from j back to i. The delay trust is then defined as [10]:

$$T_r^{ij} = min(\frac{t_{theoretical}}{t_{actual}}, 1.0) \tag{5}$$

When the actual forwarding delay is close to or shorter than the theoretical value, T_r^{ij} approaches 1.0, indicating normal behavior. When a delay attacker holds a packet for an extended period before forwarding, t_{actual} significantly exceeds $t_{theoretical}$, causing T_r^{ij} to fall proportionally. The cap at 1.0 prevents unusually fast deliveries from inflating the metric beyond its meaningful range.

3) Transmission Trust (T_t): Transmission trust detects flooding attacks by measuring the repetitiveness of packet transmissions from a neighbor. When a node receives the same packet from neighbor j multiple times, identified by the combination of original source and packet identifier carried in the TBR header, the transmission trust is penalized as:

$$T_t^{ij} = min(\frac{1}{count}, T_t^{ij}) \tag{6}$$

where $count$ is the number of times the same packet has been received from neighbor j. A legitimate node transmitting each packet exactly once maintains $T_t^{ij} = 1.0$. A flooding attacker retransmitting the same packet $count$ times drives T_t^{ij} rapidly toward zero, since $1/count$ decreases monotonically with each additional repeat. The min operator ensures that T_t^{ij} can only decrease, never recover through repeated flooding.

4) Direct Trust (T_D): The three trust metrics are aggregated into a single normalized direct trust value [10]:

$$T_D^{ij} = w_1 \cdot T_s^{ij} + w_2 \cdot T_t^{ij} + w_3 \cdot T_r^{ij} \tag{7}$$

where w_1, w_2 and w_3 are non-negative weights satisfying $w_1 + w_2 + w_3 = 1$, assigned to interaction trust, transmission trust, and delay trust respectively.

5) Indirect Trust(T_I): In the early stages of network operation, a node may have insufficient direct interaction history with some neighbors to form a reliable trust estimate. In such cases, indirect trust recommendations from common neighbor nodes prove useful as they help the evaluating node make a more informed decision with both the evaluating node i and the evaluated node j. The indirect trust is computed as the average of the direct trust values reported by all qualifying common neighbors [10]:

$$T_I^i = \frac{\sum_{k \in C} T_D^{kj}}{N_C} \tag{8}$$

msgType	orgSrc	pktID	lastSender	prevSender		
intendedNextHop	hopCount		sendTimestamp	sX	sY	sZ
senderEnergy			aboutNodeID*	newTrust*		

* only used in TBR_TRUST_UPDATE packets

Fig. 2. Structure of the Packet Header

where C is the set of common neighbor nodes that have actual interaction history with j and N_C is their count.

6) Integrated Trust (T_G): The final integrated trust combines direct and indirect trust into a single value used for relay candidate filtering and as a component of the routing and reward computations [10]:

$$T_G^i = T_D^i + \eta \cdot T_I^i \tag{9}$$

where η is a weighting coefficient that diminishes as the total number of interactions $G = a_{ij} + b_{ij}$ accumulates:

$$\eta = \frac{1}{G + 2} \tag{10}$$

When interactions are scarce, η is relatively large and indirect trust from common neighbors contributes meaningfully to the estimate. As G grows, η shrinks toward zero and T_G^i converges entirely to the direct trust T_D^i accumulated through first-hand observation. This design ensures that indirect recommendations play a corrective role only when direct evidence is limited, and that the trust evaluation becomes increasingly self-reliant as the network matures. Any neighbor whose T_G^i falls below the threshold is excluded from relay candidate consideration during routing.

C. Protocol Data Structures

TBR-QL maintains three local data structures at each node - a neighbor table, an indirect trust table, and a Q-table. Communication between nodes is carried through a unified packet header that is constructed at transmission time and immediately discarded once the relevant fields have been used to update local table.

1) Packet Header: As shown in Fig. 2, the packet header contains essential routing and trust information.

TBR-QL defines a unified packet header used across all three messages types. The three message types are TBR_DATA for normal data packets, TBR_HELLO for neighbor discovery broadcasts, and TBR_TRUST_UPDATE for trust change notifications (Indirect Trust). The fields are as follows :

- **msgType** - identifies the message type: data, hello, or trust update.
- **orgSrc, pktId** - together form a globally unique packet identifier.
- **lastSender** - address of the node currently forwarding this packet.

TABLE I
NEIGHBOR TABLE ENTRY

Field	Description
nodeId	Identity
Position	
x, y, z	3D position
distance	Euclidean distance
Energy	
initialEnergy (E_0)	First hello energy
currentEnergy (E_j)	Latest energy
energyRatio (E_j/E_0)	Energy ratio
Interaction Count	
successPackets (a_{ij})	Successful transmissions
droppedPackets (b_{ij})	Failed transmissions
Trust Metrics	
T_s	Interaction trust
T_t	Transmission trust
T_r	Delay trust
T_D	Direct trust
T_G	Integrated trust
Routing	
Ph	Routing metric
Status	
isAlive	Validity flag
lastHeardTime	Last contact timestamp

- **prevSender** - address of the node that forwarded this packet to lastSender.
- **intendedNextHop** - address of the next hop selected by the sender.
- **hopCount** - records the number of relay nodes the packet has traversed so far.
- **sendTimestamp** - the time at which the current sender transmitted this packet.
- **sX, sY, sZ** - three-dimensional position of the sender.
- **senderEnergy** - Energy of sender at time of transmission.
- **aboutNodeId, newTrust** - used only in TBR_TRUST_UPDATE packets. Identify the node whose trust changed and carry its current T_G value.

2) Neighbor Table: Each node maintains a neighbor table in which every discovered neighbor j is represented by a single entry, as shown in Table I. The table is populated during the hello phase and updated continuously throughout the data routing phase.

As illustrated in Table I, the entry is organized into seven logical groups: identity, position, energy state, interaction counters, trust metrics, routing metrics and status.

The identity and position fields are populated from hello packets and kept current with each subsequent hello exchange. The energy ratio E_j/E_0 is derived from the sender's reported current energy and its initial energy recorded at first discover, and is used directly in the Ph routing metric.

The trust fields T_t, T_t, T_r, T_D and T_G are initialized to neutral prior values and updated continuously as packets are exchanged with neighbor j, following the trust model described in Section IV-B.

3) Indirect Trust Table: Each node additionally maintains an indirect trust table, indexed by the node being evaluated and the recommending node. Each entry stores the direct trust value T_D^{kj} reported by a common neighbor k about node j, along with a timestamp recording when the recommendation arrived. This table is populated when a TBR_TRUST_UPDATE packet is received and is consulted during the indirect trust computation described in (8). Only entries from nodes that are also current neighbors of node i are considered valid recommendations, enforcing the common neighbor requirements.

4) Q table: Each node maintains a Q-table indexed by the pair (i, j) where i is the local node identifier and j is the neighbor node identifier. the Q-table stores a single scalar value $Q(j)$ representing the learned routing utility of selecting neighbor j as the next hop from node i. Q-values are initialized to the Ph composite score at neighbor discovery time and updated after each observed forwarding outcome as described in Section IV-E. The Q-table is strictly local and is never exchanged between nodes.

D. Q-Learning Enhanced Relay Selection

In TBR-QL, relay selection is entirely driven by Q-values learned through repeated packet forwarding experience. Rather than selecting the next hop based on an instantaneous score computed at each forwarding step, each node maintains a Q-value for every neighbor. The relay selection procedure is invoked each time a data packet needs to be forwarded.

1) Ph Composite Score: The composite Ph score, inherited from TBR [10], works as the foundation of the routing metric and is defined as:

$$Ph(j) = 10\lambda\frac{E_j}{E_0} + (1-\lambda)T_G^j + \frac{\sigma}{L_{ij}} \tag{11}$$

where E_j/E_0 is the residual energy ratio of neighbor j, T_G^j is its integrated trust, L_{ij} is the inter-node distance between the current node and j, λ is the weight balancing the energy and trust components, and σ is the distance scaling factor.

2) Candidate Filtering: Before any Q-value comparison is made, the node looks into its neighbor table and selects a valid candidate set by applying four criterion. A neighbor j is excluded if its depth is greater than or equal to the depth of the current node, ensuring the packet always moves towards the surface sinks. A neighbor is also excluded if its integrated trust T_G^j lies below the decided threshold θ, ensuring that nodes already identified as malicious through the trust model are never selected regardless of their Q-value. Any neighbor not recently heard from is additionally marked inactive and thus excluded. Finally if the hop count carried in the packet header exceeds the decided maximum hop limit, the packet is dropped immediately without attempting further forwarding, preventing packets from circulating indefinitely in the network. If no eligible candidate is found after applying these criterion, the packet is dropped.

3) Q-value Initialization: Before the first routing decision, each newly discovered neighbor j is assigned an initial Q-value seeded from the Ph composite score computed at discovery time:

$$Q(j) \leftarrow Ph(j) \quad \text{[at initialization]} \tag{12}$$

Since Ph is computed using the neighbor's reported energy, measured inter-node-distance, and a default trust-value, this initialization ensures that Q-values carry meaningful routing preferences showing energy and distance from the very first packet transmission. It eliminates the cold-start problem of zero-initialized Q-learning, where the agent would have no basis for distinguishing between neighbors until sufficient experience has been collected.

4) Relay Selection: Once the candidate set is formed, the node selects the next hop as the neighbor with the highest Q-value:

$$nextHop = arg \max_{j \in \text{candidates}} Q(j) \tag{13}$$

since Q-values are initialized from Ph and subsequently updated through the Bellman equation based on observed forwarding outcomes, they progressively change form their initial values as routing experience increases. A neighbor that consistently forwards packets reliably gathers a higher Q-value over time, while a malicious neighbor that drops packets, introduces delays, or floods the channel with repeated transmissions gathers a lower or negative Q-value through penalty rewards. This allows the relay selection to distinguish between neighbors that may appear identical under an instantaneous score but differ significantly in their actual routing reliability as observed over many packet transmissions. The Q-value update mechanism that drives this convergence is described in detail in Section V-E.

E. Q-value Update Mechanism

After each relay selection decision, the node observes the outcome of the forwarding event and updates the Q-value of the selected neighbor accordingly. The update follows the selected Bellman equation:

$$Q(j) \leftarrow Q(j) + \alpha \cdot [r + \gamma \cdot \max\, Q_{next} - Q(j)] \tag{14}$$

where α is the learning rate, γ is the discount factor, r is the immediate reward signal derived from the observed routing outcome, and max Q_{next} is the estimated future routing quality of neighbor j. Since each node only has access to its own Q-table and not the Q-table of its neighbors, the future state value max Q_{next} is approximated by the integrated trust T_G^j of the selected neighbor, which works as a consistent proxy for the expected quality of the path beyond j toward the sink.

1) Asymmetric Learning Rate: A key design decision in TBR-QL is the use of an asymmetric learning rate depending on the sign of the reward:

$$\alpha = \begin{cases} \alpha_{punish}, & \text{if } r < 0, \\ \alpha_{recover}, & \text{if } r \geq 0 \end{cases}$$

where $\alpha_{punish} > \alpha_{recover}$. This asymmetry ensures that malicious nodes should be penalized rapidly so that routing avoids them as early as possible, while recovery of Q-values for nodes that resume normal behavior after a transient failure should occur slowly to prevent on-off attackers from quickly regaining high Q-values during their normal behavior. As a result Q-value converges faster towards avoiding malicious nodes then trusting them again. We then discuss three cases for reward calculation.

Case 1 - Successful Forwarding

When the transmitted packet is confirmed to have been successfully relayed by neighbor j, a positive reward is computed as a weighted combination of four components:

$$r_{success} = w_t \cdot T_G^j + w_e \cdot \frac{E_j}{E_0} + w_d \cdot T_r^j + w_z \cdot r_{depth} \tag{15}$$

where w_t, w_e, w_d and w_z are non-negative weights satisfying $w_t + w_e + w_d + w_z = 1$, and the depth component r_{depth} is defined as:

$$r_{depth} = min(abs(z_i - z_j)/R, 1.0) \tag{16}$$

where z_i and z_j are the depths of the current node and neighbor j respectively, and R is the communication range. The integrated trust T_G^j rewards neighbor that have high direct and indirect trust . The energy ratio E_j/E_0 rewards neighbors that have high residual energy, showing the need to balance energy consumption across the network. The delay trust T_r^j rewards neighbors that transmit packet promptly relative to theoretical propagation delay and the depth component r_{depth} rewards neighbors that offer greater depth progress toward the surface sinks.

Case 2 - Packet Drop

When the timer expires without receiving acknowledgment from node j,a negative reward is applied:

$$r_{drop} = -1.0 * (1 + b_{ij} * 0.1) \tag{17}$$

where b_{ij} is the collective drop count for neighbor j.It is bounded by the trust threshold as when it drop below trust threshold, no node will this as next hop. The penalty therefore grows with each successive drop event, ensuring that persistent malicious nodes are penalized harshly. A neighbor that drops its first packet receives a penalty of -1.1, while a neighbor that has dropped ten packets already receives a penalty of -2.0. This scaling pushes the Q-value of malicious nodes lower at a faster rate if it continues dropping packets.

Case 3 - Flooding Attack

When the same packet is received form neighbor j more than once, a Q-value update is triggered immediately. The reward is derived directly from the current transmission trust T_t^{ij}:

$$r_{repeat} = min(T_t^{ij} - 0.5, 0) \tag{18}$$

As T_t^{ij} decreases with each additional repeat, r becomes progressively more negative, with the clamp at zero ensuring it is never positive. The punishment learning rate α_{punish} is applied in the Bellman update, ensuring that the Q-value of a flooding neighbor collapses rapidly with each detected repeat rather than degrading gradually.

Summary of Q-Update Cases

$$Q(j) \leftarrow \begin{cases} Q + \alpha_{\text{recover}} \cdot \left[r_{\text{success}} + \gamma \cdot T_G^j - Q \right] \\ \quad \text{successful forward packet} \\ Q + \alpha_{\text{punish}} \cdot \left[r_{\text{drop}} + \gamma \cdot T_G^j - Q \right] \\ \quad \text{packet drop} \\ Q + \alpha_{\text{punish}} \cdot \left[r_{\text{repeat}} + \gamma \cdot T_G^j - Q \right] \\ \quad \text{repeated packet} \end{cases}$$

Over successive packet transmissions,Q-values for reliable neighbors converge upward driven by consistent positive rewards, while Q-values for malicious neighbors collapse downward driven by negative rewards applied at the faster learning rate. This convergence is what causes the relay selection in Section V-D to progressively avoid malicious neighbors without requiring any explicit classification of attack type.

V. Simulation and Performance Analysis

In this section, simulation results are provided to verify and analyze the performance of the proposed TBR-QL protocol. TBR-QL is evaluated under five attack scenarios - packet drop, replay , flooding , mixed and mixed on-off - at three attacker densities of 10%, 20%, and 30% of the relay node population. Performance is additionally compared against the baseline TBR protocol under two data rate configurations, demonstrating that while TBR suffers a significant degradation in packet delivery ratio when network network load is doubled, TBR-QL maintains stable performance owing to the ability of Q-values to accumulate routing evidence from increased packet interactions and more reliably distinguish malicious nodes from congested nodes that can be used as relay in future. Finally, the convergence behavior of Q-values and Ph scores is analyzed over the simulation duration to provide evidence that the Q-leaning mechanism correctly and progressively differentiates reliable neighbors from malicious ones, supporting the PDR(Packet Delivery Ratio) improvements observed in the performance results.

A. Simulation Setup

All simulation are conducted on a 660-node three-dimensional deployment using NS-3.41 [15] with the AquaSim-NG underwater network module [16]. The medium access control layer employs a CSMA-based protocol following the approach adopted in VBF [17]. The experimental parameters are listed in Table II. The performance results reported in Sections V-B and V-C are obtained at data rate of 228 bits/s.

Three performance metrics are used throughout this section. These are defind as follows-: First, Packet Delivery Ratio which is defined as-:

$$PDR = \frac{P_{sink}}{P_{sent}} * 100\% \tag{19}$$

where P_{sink} is the number of packets successfully received at any sink node and P_{sent} is the total number of packets transmitted by all source nodes. The Average End-to-End Delay is defined as:

$$\bar{D} = \frac{1}{P_{sink}} \sum_{k=1}^{P_{sink}} (t_{recv}^k - t_{sent}^k) \tag{20}$$

where t_{recv}^k and t_{sent}^k are the reception and transmission timestamps of the k -th successfully delivered packet respectively. The Effective Energy is defined as :

$$E_{eff} = \frac{E_{total}/N}{P_{sink}} * 100 \quad \text{(J)} \tag{21}$$

where E_{total} is the total energy consumed by the network, N is the total number of nodes, and P_{sink} is the number of packets received at the sink. This metric captures the true energy cost of delivering each packet to the sink, penalizing protocols that consume energy without matching packet delivery. A lower value of E_{eff} indicates better energy efficiency relative to the number of packets successfully delivered.

TABLE II
Simulation Parameters

Category	Parameter	Value
Network Topology	Network Volume (m^3)	$666 \times 666 \times 1000$
	Number of Source Nodes	10
	Number of Relay Nodes	600
	Number of Sink Nodes	50
Energy and Power	Initial Node Energy, E_0 (J)	10,000
	Transmission Power, P_{tx} (W)	0.660
	Reception Power, P_{rx} (W)	0.395
	Idle Power, P_{idle} (W)	0.008
Communication	Packet Size (bytes)	50
	Communication Range, R (m)	180
	Sound Velocity, V_s (m/s)	1500
	Maximum Hop Count	30
Trust Model	Trust Threshold, θ	0.2
	Initial T_s, T_t, T_r	0.5
	Initial T_D	0.5
	Initial T_I	0
	Interaction Trust Weight, w_1	0.25
	Transmission Trust Weight, w_2	0.5
	Delay Trust Weight, w_3	0.25
Routing	Energy Weight, λ	0.2
	Distance Weight, σ	0.5
	Trust Weight, w_t	0.5
	Energy Weight, w_e	0.1
	Delay Weight, w_d	0.3
	Depth Weight, w_z	0.1
Q-Learning	Learning Rate (Punishment), α_{punish}	0.6
	Learning Rate (Recovery), $\alpha_{recover}$	0.1
	Discount Factor, γ	0.7

TABLE III
PACKET DELIVERY RATIO (%) OF TBR-QL UNDER ATTACK SCENARIOS

Attack	10% Attackers	20% Attackers	30% Attackers
Packet Drop	91.32%	88.75%	87.51%
Replay	95.13%	94.35%	89.90%
Flooding	95.07%	95.70%	91.58%

TABLE IV
AVERAGE E2E DELAY OF TBR-QL UNDER ATTACK SCENARIOS

Attack Scenario	10% Attackers	20% Attackers	30% Attackers
Packet Drop	2.18 s	2.11 s	1.99 s
Replay	2.40 s	2.64 s	3.37 s
Flooding	2.22 s	2.16 s	2.11 s

B. Performance of TBR-QL Under Individual Attacks

This section evaluates the performance fo TBR-QL under three individual attack scenarios - packet drop, replay, and flooding - at attacker densities of 10%, 20%, 30% of the relay node population.

1) Packet Delivery Ratio Analysis: Table III presents the packet delivery ratio of TBR-QL across all three attack scenarios and attacker densities. TBR-QL consistently maintains high PDR across all conditions, with values ranging from 87.51% to 95.7%. A notable observation is that increasing the attacker density from 10% to 30% does not cause a significant degradation in PDR - the largest drop across any single attack is 5.50% for replay Attack, from 95.13% at 10% to 89.90% at 30%. This stability demonstrates that the Q-learning mechanism effectively identifies and progressively avoids malicious neighbors regardless of how many are present in the network. Among the three attack types, packet drop causes the lowest PDR across all densities, as dropping nodes directly eliminate packets from the network. Replay and Flooding attacks maintain higher PDR because packets are eventually delivered rather then getting eliminated entirely, and thus Q-value penalty for these behaviors is correspondingly more gradual.

2) End to End Analysis: Table IV presents the average end-to-end delay under the three attack scenarios. Delay values remain within the range of 1.99 s to 3.37 s across all conditions. The replay attack produces the highest delays, increasing from 2.40 s at 10% to 3.37 s at 30%, as these attackers explicitly hold packets before forwarding, directly increasing the measured delivery time. Packet drop attacks show a decrease in average delay from 2.18 s at 10% to 1.99 s at 30%, which occurs because dropped packets are excluded from the delay calculation - only successfully delivered packets contribute to the average, thus reducing end to end delay. Flooding attacks show the most stable delay behavior across densities, ranging narrowly from 2.22 s to 2.11 s.

TABLE V
EFFECTIVE ENERGY OF TBR-QL UNDER ATTACK SCENARIOS

Attack Scenario	10% Attackers	20% Attackers	30% Attackers
Packet Drop	3.5876 J	3.6507 J	3.5426 J
Replay	3.4859 J	3.6027 J	3.8571 J
Flooding	3.7294 J	3.6656 J	3.6391 J

TABLE VI
TBR-QL PERFORMANCE UNDER MIXED ON-OFF ATTACK

Performance Metric	Value
Packet Delivery Ratio	85.12%
Average End-to-End Delay	2.29 s
Effective Energy	3.9042 J

3) Effective Energy Analysis: Table V presents the value of effective energy under three attack scenarios. The values range from 3.4859 J to 3.8571 J, indicating consistent energy consumption in all types and densities of attacks. The replay attack shows the most significant increase with attacker density, rising from 3.4859 J at 10% to 3.8571 J at 30%, caused by the additional retransmissions and routing overhead incurred as more delay attackers introduce path-level latency and force the protocol to explore alternative routes. The packet drop scenario remains relatively stable between 3.5426 J and 3.6507 J across densities. Flooding attacks show a slight decrease in effective energy from 3.7294 J at 10% to 3.6391 J at 30%. Thus, this confirms that TBR-QL maintains energy-efficient routing even when the percentage of malicious nodes in the network increases.

C. Performance Under Mixed On-Off Attack

The mixed on-off attack is the strongest attack. Thirty percent of relay nodes are active simultaneously across all three attack types - 10% dropping packets, 10% holding them back before forwarding, and 10% flooding the channel with repeated copies. Every one of those nodes also operates under the on-off mechanism, randomly entering and exiting the attacking phase. A node dropping packets five seconds ago might be forwarding packets correctly right now, making it genuinely difficult to identify who is malicious and who is dropping packets because of channel conditions.

Table VI gives the result of simulation with this attack. In it TBR-QL achieves 85.12% PDR which is lower compared to any other attack because it is most difficult to identify for malicious nodes when malicious nodes is switching on/off. Due to this Q-value takes more time to converge to lower value for attacking nodes and thus it leads to more packet dropping.

D. Comparative Analysis: TBR vs TBR-QL

This section compares TBR-QL directly against baseline TBR under a mixed attack scenario with 30% attacking nodes,

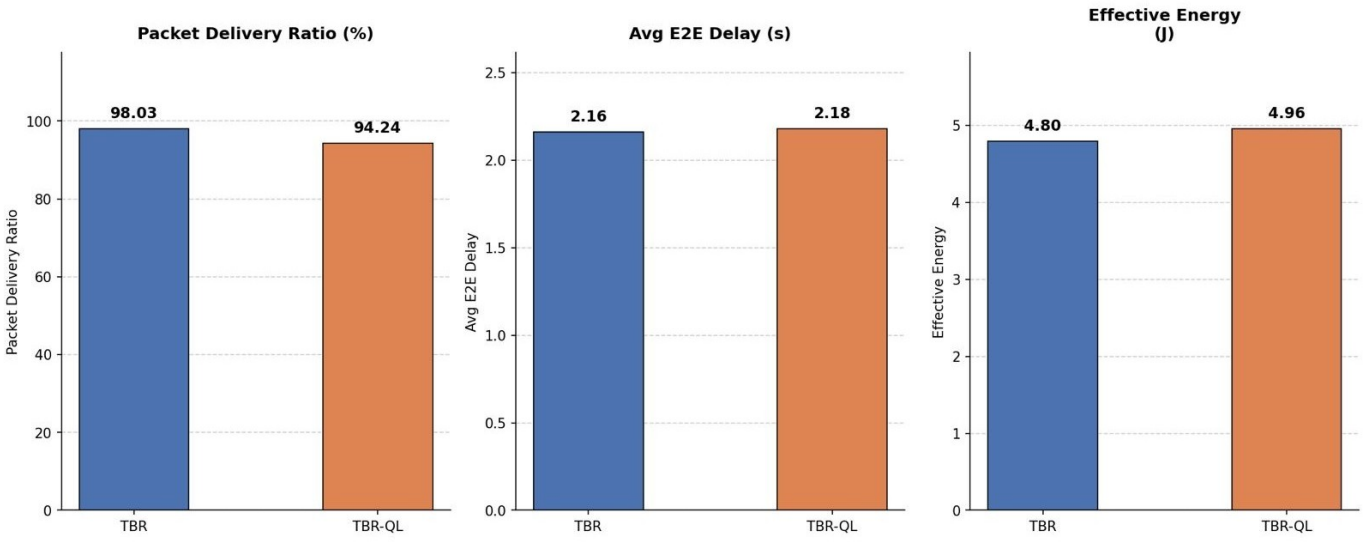

Fig. 3. TBR vs TBR-QL Performance at 114 bits/s

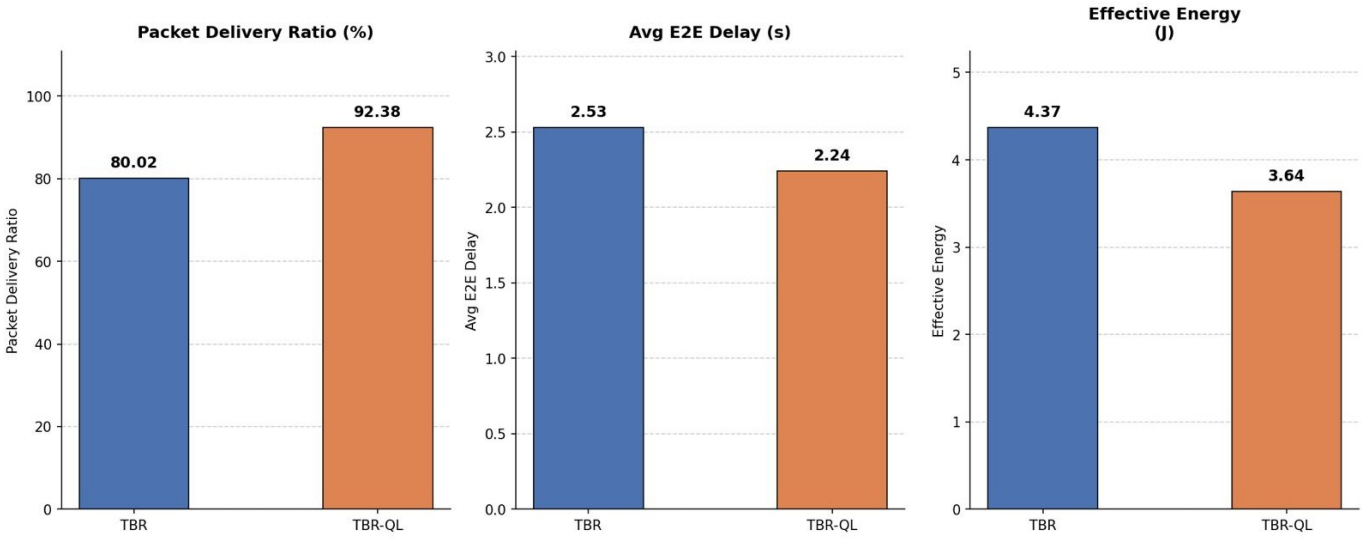

Fig. 4. TBR vs TBR-QL Performance at 228 bits/s

at two data rates. Fig. 3 shows results at 114 bits/s and Fig. 4 at 228 bits/s.

At the 114 bits/s, TBR achieves 98.03% PDR against TBR-QL's 94.24%. at lower data rate, the network generates fewer packets per unit time, which means Q-values accumulate routing evidence more slowly. The Ph-seeded initialization gives a reasonable starting point, but without enough packet interactions to drive meaningful divergence between malicious and good nodes, TBR-QL's selection effectively behaves like a noisier version of Ph-based routing during much of the simulation period. TBR, computing Ph fresh at every hop from current trust and energy values, has no convergence period to wait through. Under lower channel load the congestion problem is also less severe so there are no packet drops due to channel being busy thus leading to very high PDR.

Doubling the data rate changes things considerably. TBR falls to 80.02% PDR; TBR-QL holds at 92.38%.

The root cause of TBR's decline is not the attackers alone. At 228 bits/s, channel contention rises and legitimate relay nodes begin dropping packets due to queue overflow and collisions. From TBR's perspective, this looks identical to a packet drop attack: b_{ij} climbs, T_s falls, and the node eventually drops below the trust threshold and gets cut from routing candidates. TBR cannot tell the difference between a node that is overwhelmed and one that is malicious, so it removes both. As confirmed by the Ph convergence analysis in Section V-E, the Ph scores of legitimate and malicious nodes remain clustered within a narrow band under higher channel

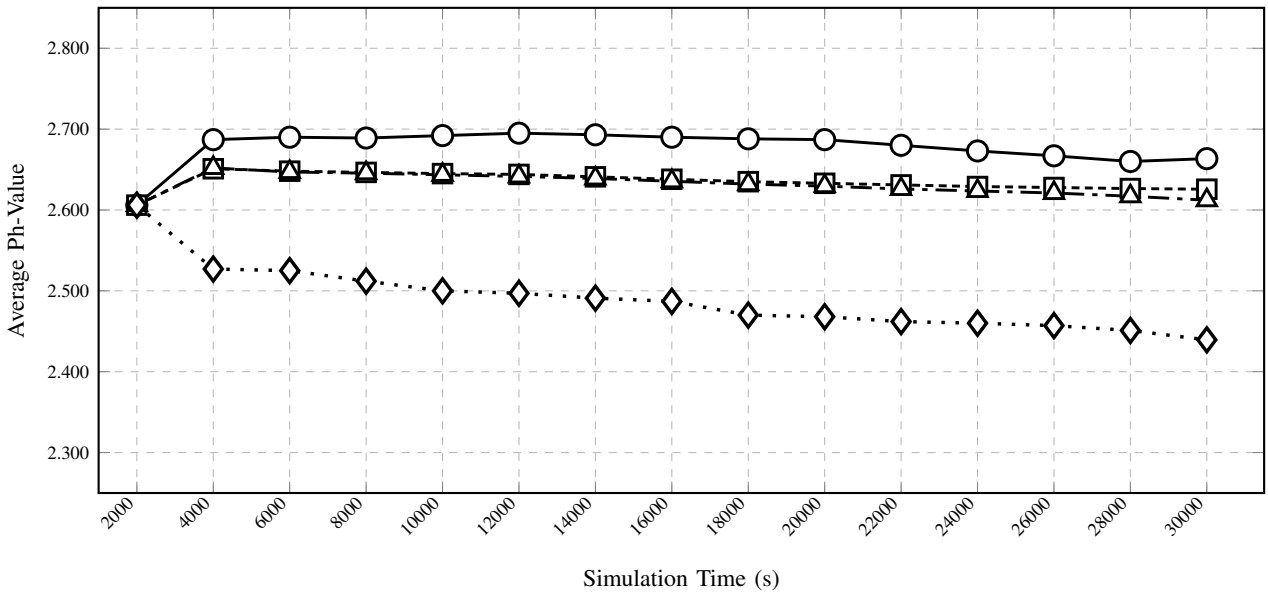

Fig. 5. Ph-Value Convergence: Normal vs Attacker Nodes

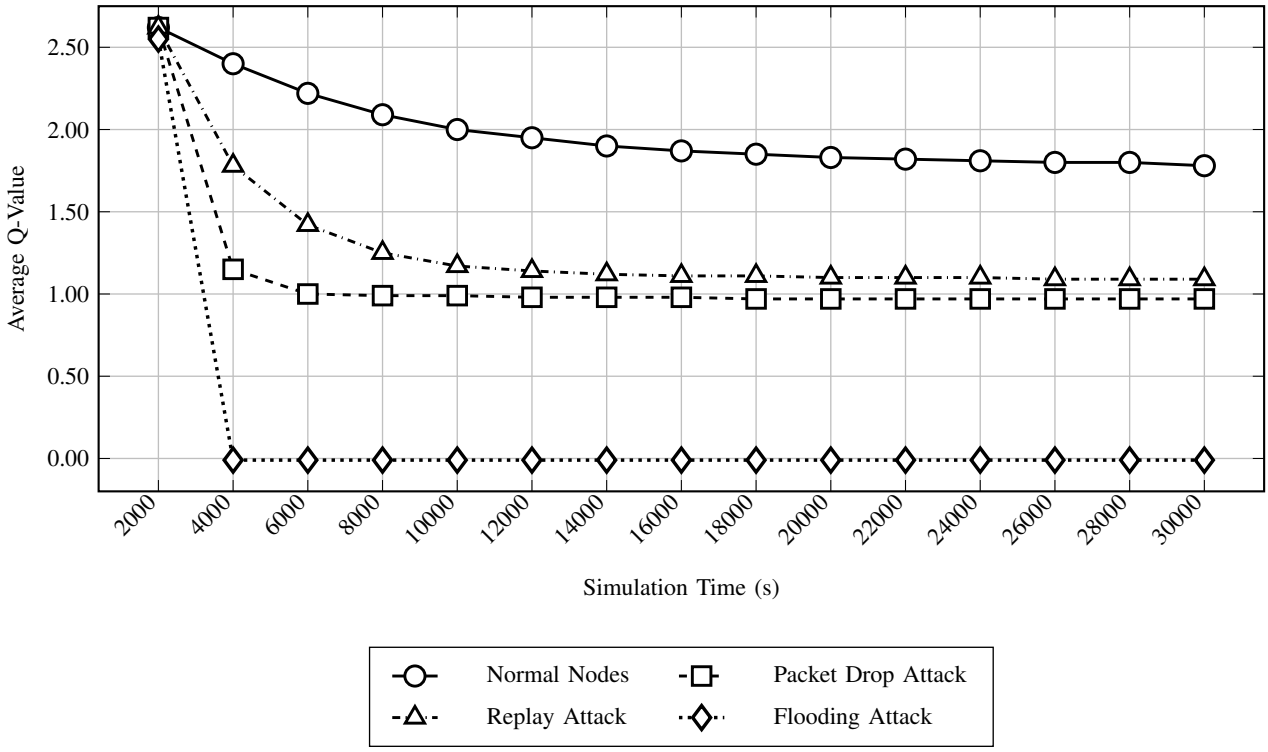

Fig. 6. Q-Value Convergence: Normal vs Attacker Nodes

load, making it difficult for TBR to differentiate between legitimate and malicious nodes.

TBR-QL avoids this problem through its Q-value update mechanism, which tracks actual forwarding outcomes over time rather than relying on instantaneous scores. A legitimate node that drops packets under congestion recovers and starts forwarding correctly once load eases, gradually accumulating positive rewards and keeping its Q-value at an acceptable level. A malicious dropper never recovers because its drops are intentional and persistent, so its Q-value keeps declining regardless of channel conditions. Over the course of simulation, the two populations diverge clearly in Q-value space even when they remain cloes in Ph space as seen in Section V-E.

There is a second mechanism at work. TBR-QL's reward

function includes the energy ratio E_j/E_0 as an explicit component. Nodes under heavy load consume energy faster and report lower energy ratios, which reduces the reward they generate and causes the Q-learning mechanism to gradually route traffic away from them toward less loaded relay nodes with more available energy. This produces a load-balancing effect that TBR's Ph metric cannot replicate: Ph incorporates the energy ratio as a fixed term but receives no feedback from forwarding outcomes, so it cannot adapt routing away from congested nodes dynamically. The practical consequence is that TBR-QL reduces packet drops from congestion as well as from malicious behavior, while TBR addresses neither effectively at higher data rates.

The delay falls from 2.53 s to 2.24 s and effectively energy drops from 4.37 J to 3.64 J, both consistent with fewer failed routing decisions and more balanced traffic distribution across the network.

The contrast between the two protocols across data rates captures the practical advantage of the Q-learning approach concisely. When the data rate doubles from 114 bits/s to 228 bits/s under mixed attack conditions. TBR-QL's PDR drops by only 2% but under the same conditions, TBR drops by 18%. This difference in sensitivity to network load demonstrates that TBR-QL delivers consistent performance across varying traffic conditions, while TBR's selection degrades significantly as channel utilization increases.

E. Q-value and Ph-Value Convergence Analysis

This section analyzes the convergence behavior of Q-values and Ph scores over the simulation duration to explain why Q-learning produces better relay selection than Ph-based routing under attack conditions. Fig. 5 shows Ph-value convergence and, Fig. 6 shows Q-value convergence, both measured as the average score assigned to each node category every 2000 seconds from $t = 2000\,\text{s}$ to $t = 30000\,\text{s}$.

Fig. 5 reveals a fundamental limitation of Ph-based selection. At the start of the data phase, all four node categories : normal, packet drop, replay and flooding carry nearly identical Ph scores as energy levels and trust values are uniformly initialized for all nodes. The curves for normal nodes, packet drop attackers, and replay attackers remain so close throughout the simulation that distinguishing them for routing decisions is not feasible. Flooding attackers show a more visible decline, as the transmission trust T_t degrades with each repeated packet and propagates through t_G int Ph, but even this separation strays narrow enough to be unreliable under elevated channel load as shown in Section V-D. What is more prominent is how little changes over the following 28000 seconds. The underlying issue is that Ph is computed from the current state of a neighbor: its energy, trust, and distance at the moment of selection with no memory of how that neighbor has behaved across previous packet transmissions. Under stress, that information is simply not enough.

Fig. 6 presents a different picture. Q-values begin at the same level as Ph scores they are initialized form Ph at neighbor discovery, but the four curves diverge sharply within the first few thousand seconds and maintain clear separation through the remainder of the simulation. Flooding attackers collapse almost immediately, driven by rapid negative Q-updates each time a repeated packet is detected. Packet drop attackers stabilize at a consistently low level, held there by the negative reward that fires every time after not receiving an acknowledgment. Replay attackers settle at an intermediate level because they do eventually forward packets and collect some positive reward, but below normal nodes because of the delay penalty. Normal nodes decline gradually as the Bellman update reaches equilibrium, but the margin between them and all the three attacker categories grows wide enough, well before the simulation ends, that channel noise cannot meaningfully narrow it, as shown in Section V-D. Taken together, the two convergence graphs provide the mechanistic explanation for the PDR results reported throughout this section. Ph-based routing has no access to forwarding history and cannot sustain the separation between node categories needed to route reliably around attackers when the channel is congested. Q-values earn that separation through accumulated forwarding experience, and once established it is durable. The implication is that TBR-QL becomes a more effective routing protocol as the network operates and the conditions that degrade TBR most severely are precisely the conditions under which TBR-QL's learned routing memory provides the greatest advantage.

VI. Conclusion

This article presented TBR-QL, a Q-learning enhanced trust-based routing protocol for underwater wireless sensor networks. TBR-QL addresses a key limitation of existing trust-based routing schemes, where instantaneous trust scores often fail to distinguish malicious packet dropping from congestion-induced losses under high network load. Instead of recomputing routing decisions independently at every hop, each node maintains learned utility(Q value) for its neighbors based on observed forwarding behavior over time. The Q-learning update incorporates trust reliability, residual energy, propagation delay, and depth advancement toward the sink, while an asymmetric learning strategy penalizes malicious behavior faster than it rewards recovery. An autonomous on-off attack model was also introduced to evaluate resilience against adaptive insider attacks that alternate between cooperative and malicious behavior. Simulation results demonstrated that TBR-QL consistently outperforms baseline TBR across different attack scenarios and attacker densities, with the largest improvements observed under heavy traffic conditions where conventional Ph-based routing degrades significantly.

VII. Future Work

Several directions remain open for future investigation. The current deployment assumes fixed node positions, which is a reasonable simplification for infrastructure monitoring application but does not capture scenarios involving autonomous underwater vehicles or drifting sensor nodes. Extending TBR-QL to support node mobility through periodic neighbor table

updates triggered by hello broadcasts would allow Q-values to adapt to changing network topology, maintaining routing quality as neighbor relationships evolve over time. Additionally, the flat network architecture used in this work treats all relay nodes uniformly, which may become inefficient at larger network scales. Incorporating a clustering mechanism, where cluster heads aggregate trust information and coordinate routing within their regions, could reduce per-node Q-table overhead and improve scalability without sacrificing the security benefits of the Q-learning relay selection. Finally, evaluating TBR-QL against deep reinforcement learning approaches that use neural network function approximation in place of tabular Q-values represents a natural extension for networks too large for per-neighbor Q-table storage.

REFERENCES

[1] M. Jahanbakht, W. Xiang, L. Hanzo, and M. Rahimi Azghadi, "Internet of Underwater Things and big marine data analytics: A comprehensive survey," IEEE Commun. Surveys Tuts., vol. 23, no. 2, pp. 904–956, Secondquarter 2021.

[2] Z. Liu, Y. Luo, L. Pu, M. Zuba, and Z. Peng, "Energy-efficient data collection scheme based on value of information in underwater acoustic sensor networks," IEEE Internet Things J., vol. 11, no. 4, pp. 6621–6635, Feb. 2024.

[3] M. Stojanovic and J. Preisig, "Underwater acoustic communication channels: Propagation models and statistical characterization," IEEE Commun. Mag., vol. 47, no. 1, pp. 84–89, Jan. 2009.

[4] Z. Liu, Y. Luo, L. Pu, and Z. Peng, "AUV-aided hybrid data collection scheme based on value of information for Internet of Underwater Things," IEEE Internet Things J., vol. 9, no. 24, pp. 25948–25961, Dec. 2022.

[5] H. A. A. Al-Kashoash, H. Kharrufa, Y. Al-Nidawi, and A. H. Kemp, "Congestion control in wireless sensor and 6LoWPAN networks: Toward the Internet of Things," Wireless Netw., vol. 25, no. 8, pp. 4493–4522, Nov. 2019.

[6] G. Han, J. Jiang, L. Shu, and M. Guizani, "An attack-resistant trust model based on multidimensional trust metrics in underwater acoustic sensor networks," IEEE Trans. Mobile Comput., vol. 14, no. 12, pp. 2447–2459, Dec. 2015

[7] J. Du, G. Han, C. Lin, and M. Martínez-García, "ITrust: An anomaly-resilient trust model based on isolation forest for underwater acoustic sensor networks," IEEE Trans. Mobile Comput., vol. 21, no. 12, pp. 4425–4438, Dec. 2022.

[8] Y. Su, S. Ma, H. Zhang, Z. Jin, and X. Fu, "A redeemable SVM-DS fusion-based trust management mechanism for underwater acoustic sensor networks," IEEE Sensors J., vol. 21, no. 19, pp. 21977–21989, Oct. 2021.

[9] A. Signori, F. Campagnaro, I. Nissen, and M. Zorzi, "Channel-based trust model for security in underwater acoustic networks," IEEE Internet Things J., vol. 9, no. 20, pp. 20189–20201, Oct. 2022.

[10] Z. Liu, Z. Sun, J. Su, Y. Yuan, and X. Guan, "TBR: Secure routing design for UWSN based on trust management models," IEEE Internet Things J., vol. 12, no. 17, pp. 36674–36685, Sep. 2025.

[11] J. Jiang, G. Han, L. Shu, M. Guizani, and J. J. P. C. Rodrigues, "Controversy-adjudication-based trust management mechanism in the Internet of Underwater Things," IEEE Internet Things J., vol. 10, no. 3, pp. 2603–2614, Feb. 2023.

[12] H. Yan, Z. Shi, and J.-H. Cui, "DBR: Depth-based routing for underwater sensor networks," in Proc. IFIP Networking Conf., Singapore, 2008, pp. 72–86.

[13] F. Li, Y. Xu, Z. Sun, and X. Guan, "SDN-QLTR: Q-learning-assisted trust routing scheme for SDN-based underwater acoustic sensor networks," IEEE Internet Things J., vol. 11, no. 6, pp. 10682–10694, Mar. 2024.

[14] M. Hosseinzadeh, A. Rezazadeh, and M. R. Meybodi, "QLTM: A Q-learning-based trust model for underwater acoustic sensor networks," Ad Hoc Netw., vol. 163, Art. no. 103706, 2025.

[15] G. F. Riley and T. R. Henderson, "The NS-3 network simulator," in Modeling and Tools for Network Simulation, K. Wehrle, M. Günes, and J. Gross, Eds. Berlin, Germany: Springer, 2010, pp. 15–34.

[16] P. Xie, Z. Zhou, Z. Peng, J.-H. Cui, and Z. Shi, "Aqua-Sim: An NS-2 based simulator for underwater sensor networks," in Proc. IEEE OCEANS, Biloxi, MS, USA, 2009, pp. 1–7.

[17] H. Yan, Z. J. Shi, and J.-H. Cui, "VBF: Vector-based forwarding protocol for underwater sensor networks," in Proc. IFIP Networking Conf., Coimbra, Portugal, 2006, pp. 1216–1221.